

# PILOT

Portable Intelligent Listener  
for Open Tasking

---

Voice AI Capstone — Grid Dynamics

Phase 1 · 2025–2026



PIPELINE

# What is PILOT?

A fully local, real-time voice agent — no cloud, no paid API keys required.

100%

Local Inference

All models run on-device

40+

Languages

Auto language detection + TTS

0

Paid API Keys

Open-source inference only

PILOT listens continuously, distinguishes speakers, verifies identity, and gates sensitive actions behind voice authorization. **Built to be provider-swappable** — swapping to a cloud API requires only an env-var change, not a refactor.

# Phase 1 Pipeline

Six modular components, each independently swappable.



## DATA FLOW

### Speaker-Aware Context

Diarization labels map anonymous speakers to enrolled identities. The LLM receives labelled transcript: "Alice: ...", "Bob: ..."

### VAD-Gated Recording

WebRTC VAD (30 ms frames) detects speech onset with a 5-frame rolling window to prevent false triggers from noise.

### GPU Auto-Detection

Both pyannote and Faster-Whisper auto-detect CUDA availability. Falls back gracefully to CPU on laptops.

# Voice Enrollment & Verification

WeSpeaker ResNet34-LM embeddings stored in SQLite.

## Record 3 Samples

Browser UI (enrollment.html) captures audio via MediaRecorder API

## Compute Embeddings

WeSpeaker ResNet34 generates 256-dim speaker embedding per sample

## Average & Normalize

Mean embedding computed, L2-normalized to unit vector, stored in SQLite

## Cosine Verification

Incoming audio compared via cosine similarity. Threshold: 0.55

### SQLite Schema (~/.pilot/pilot.db)

#### profiles

- id, name, slug
- enrolled\_at, sample\_count
- embedding (BLOB, float32)

#### verification\_log

- id, checked\_at
- matched\_slug, score
- verified (0 | 1)

# Module Reference

Six core modules — each a clean, independent unit.

[recorder.py](#)

## VAD-Gated Capture

sounddevice + WebRTC VAD (30ms frames). Onset detection via 5-frame rolling window. Returns float32 mono ndarray.

[stt.py](#)

## Whisper ASR

Faster-Whisper (CTranslate2). Accepts ndarray, bytes, or path. Built-in VAD filter. beam\_size=5.

[diarization.py](#)

## Speaker Diarization

pyannote 3.1 pipeline. Resolves SPEAKER\_00/01 labels to enrolled names via cosine similarity.

[speaker.py](#)

## Verification & Enroll

WeSpeaker ResNet34-LM. enroll() / verify() / identify\_speaker(). Full SQLite audit log.

[llm.py](#)

## Ollama Chat Client

ConversationMemory with rolling history. chat() blocking + stream\_chat() generator. reset\_conversation().

[tts.py](#)

## Multilingual TTS

Edge-TTS primary (internet). Kokoro ONNX fallback (offline). Auto language detect via langdetect.

# Multilingual Text-to-Speech

Auto-detects language and routes to the right neural voice.

Primary

## Edge-TTS

- ✓ Microsoft neural voices
- ✓ 40+ languages supported
- ✓ Indian languages: Tamil, Hindi, Telugu, Kannada, Malayalam, Bengali, Gujarati, Marathi, Urdu
- ✓ Requires internet connection

Fallback

## Kokoro ONNX

- ✓ Fully offline inference
- ✓ English-only
- ✓ Auto-downloaded to ~/.cache/kokoro/
- ✓ ONNX runtime — no PyTorch needed

# Setup & Running PILOT

Three run modes, all configurable via .env

## Prerequisites

Python 3.10+ (pyannote + faster-whisper)

Ollama (ollama.com → ollama pull qwen3:8b)

ffmpeg (MP3→PCM decode, WebM→WAV for enrollment)

HuggingFace token (first model download only)

```
# .env — key settings
STT_MODEL=base
LLM_MODEL=qwen3:8b
TTS_ENGINE=edge
SPEAKER_THRESHOLD=0.55
SPEAKER_HF_TOKEN=hf_...
```

## Run Modes

```
python voice_agent.py
```

Interactive loop · mic → verify → diarize → LLM → TTS

```
python voice_agent.py --once
```

Single turn, then exit

```
python voice_agent.py --text
```

Keyboard input, voice output — no mic needed

```
python voice_agent.py --no-verify
```

Skip speaker verification (dev mode)

```
python voice_agent.py --no-diarize
```

Single-speaker mode, faster

```
python enroll_server.py
```

Start enrollment UI at localhost:5050

Voice commands: "exit" to quit · "reset" to clear memory

# 6-Week Plan & Phase 2 Vision

Phase 1 is the kernel — Phase 2 adds streaming, concurrency, and demos.



## Phase 2 Component Upgrades

| Component   | Phase 1 (Current)      | Phase 2 Target                        |
|-------------|------------------------|---------------------------------------|
| VAD         | WebRTC (energy-based)  | Smart Turn v3 (semantic endpointing)  |
| Diarization | pyannote 3.1 (offline) | Sortformer (NVIDIA, streaming)        |
| LLM         | Single Ollama model    | Front LLM (router) + Background agent |
| TTS         | Edge-TTS + Kokoro      | Streaming chunks → browser playback   |

# Planned Demo Scenarios

Three scenario scripts that stress different parts of the system.



Week 5

## PPT Copilot

Voice-driven slide navigation via reveal.js  
WebSocket bridge. Front LLM classifies "next  
slide" intent vs conversational "and next...".  
Commands: next/prev, jump by title index.

*Stress test: Distinguishing commands from mid-sentence filler words*



Week 5

## Standup / Jira

Multi-speaker session with enrolled roles.  
Developers append ticket notes; managers alone  
flip workflow states. Interruptions mid-tool run  
visibly cancel or rollback partial writes.

*Stress test: RBAC keyed on probabilistic speaker ID + confirmation gates*



Week 6

## Customer Care Intake

Duplex capture: CSR + customer, sometimes  
overlapping. Background agent fills canonical  
fields while only CSR identity may trigger  
submission. Customer speech never triggers  
writes.

*Stress test: Spoofed intents — customer mimicking CSR phrasing*

# PILOT

Phase 1 — Proof of Concept

---

All six components are functional, tested, and provider-swappable. The kernel is ready for Phase 2 streaming and concurrency work.

## What's been built

VAD-gated microphone capture with WebRTC

Local Faster-Whisper STT (CPU + GPU)

Multi-speaker diarization via pyannote 3.1

Voice enrollment + cosine verification (SQLite)

Ollama LLM with rolling conversation memory

Multilingual TTS · Edge-TTS + Kokoro fallback

Browser-based enrollment UI + Flask server

All config in `.env` — swap providers via env vars